

OpenLearn Next V2

插件开发、内核 API 及系统指令全集指南

面向全栈开发者与架构维护者的生产级核心参考手册

核心研发团队发布

文档版本: v2.3.0 • 发布时间: 2026年7月 • 技术栈: TS / ESM / Sqlite / Node Worker

一、 插件系统整体架构

OpenLearn Next V2 采用高安全性、进程级隔离的多核沙箱架构。插件系统作为系统的核心扩展机制，支持动态热插拔、能力声明、资源追踪及安全限制。

💡 核心架构理念

架构以**最小特权原则 (Principle of Least Privilege)** 为基础。插件默认没有任何系统调用与内核接口访问权限。所有的权限提升、数据库访问以及服务交互，都必须通过清单声明并在激活时由系统统一授予或配置。

1. 运行模式 (Execution Modes)

- **内联模式 (Inline Mode):** 插件代码直接在主线程中执行，适用于系统级内置插件（如 `VFS`、`Builtin` 核心插件）。具有极高执行效率，但隔离等级较低。
- **沙箱 Worker 模式 (Worker Mode):** 插件通过 Node.js `Worker Threads` 在隔离的独立线程中加载运行。该模式下，插件无法执行未经授权的系统命令或破坏内核内存结构，所有与系统底座的交互必须通过 RPC 通道进行。

2. 内核生命周期与安全网关

1. **安装与验证 (Installation):** 插件以 `.zip` 压缩包上传，由 `PluginHost` 校验 `manifest.json` 的合法性，并解压至隔离的磁盘目录。
2. **沙箱激活 (Activation):** 系统读取清单中的能力声明 (Capabilities)，生成唯一的 UUID 作为插件运行时 ID。若插件配置为 `worker` 模式，`WorkerManager` 会注入专用的 RPC 引导模板并生成独立的沙箱上下文。
3. **自动卸载与清理 (Deactivation):** 插件停用时，`ResourceTracker` 会自动回收该插件已注册的画板元素、临时处理器与事件订阅，并安全销毁 Worker 线程以防内存泄漏。

此外，系统内置了**审批网关 (Approvals Gateway)**。当非管理员角色的用户触发高风险操作指令时，该操作将被挂起等待教师/管理员手动审批；而如果是管理员角色用户进行的高危操作，则无需审批，系统会自动通过并继续运行。

二、 插件上下文设计与 API 接口

当插件的 `activate(ctx)` 方法被调用时，内核会传入一个沙箱化的 `PluginContext` 实例。该实例是插件与内核交互的唯一合法窗口。

1. PluginContext 接口定义

```
export interface PluginContext {
  readonly pluginId: string;           // 运行时分配的唯一 UUID
  readonly manifest: Manifest;        // 插件解析后的清单配置

  readonly services: {
    readonly commandBus?: ICommandBusService; // 指令总线，用于下发系统控制
    readonly actionRegistry?: IActionRegistryService; // 动作注册，用于发布扩展指令
    readonly eventBus?: IEventBusService; // 事件总线，订阅系统事件
    [token: string]: any; // 其他已被授权注入的内核服务
  };

  resolve<T>(token: string | Token<T>): Promise<T>; // 动态解析已被授权的依赖注入服务
  require(moduleName: string): any; // 安全加载被允许共享的外部模块（如 xlsx, uuid）

  readonly db: {
    ensureTable(tableName: string, schema: string): Promise<void>; // 声明并创建隔离的插件自建表
    table(tableName: string): string; // 解析为隔离物理表名
    dropAllTables(): Promise<void>; // 清空插件名下的所有自建表
  };
}
```

2. 内核共享模块支持列表

模块名称	说明	适用场景
xlsx	SheetJS 电子表格操作库	数据导出、报表计算、作业明细生成等
uuid	标准 UUID 生成工具	生成高碰撞防护的唯一资源与主键 ID
jspdf	PDF 文件生成工具	生成精美的课后总结报告 PDF
jspdf-autotable	jspdf 表格插件	在 PDF 中排版复杂的多列数据表
react-markdown	Markdown 渲染器	用于插件端 UI 的 Markdown 编译渲染

三、 清单文件完全指南 (manifest.json)

每个插件压缩包中必须在根目录包含一个 `manifest.json` 文件，用于声明插件的基本元信息、入口配置以及安全权限。

1. 属性字段定义说明

字段名称	数据类型	是否必填	描述与格式规范
<code>id</code>	String	是	插件的唯一标识符。 命名规范： 第三方插件必须以 <code>ext-</code> 开头，以和内置核心插件区分开（如 <code>ext-quiz-generator</code> ）。
<code>name</code>	String	是	人类可读的插件名称（可包含中文字符）。
<code>version</code>	String	是	SemVer 规范版本号： 符合 <code>MAJOR.MINOR.PATCH</code> 格式（如 <code>1.2.4</code> ），用于热更新版本比对。
<code>entry</code>	String	是	插件运行的主入口 JS 文件相对路径（如 <code>index.js</code> ）。支持被 ESBUILD 打包后的单文件构建产物。
<code>author</code>	String	否	插件创作者的名称。
<code>homepage</code>	String	否	插件项目的官方主页 URL。
<code>capabilitiesProposed</code>	String[]	否	能力声明集合： 插件希望被授予的权限列表（如 <code>["whiteboard:write", "vfs:write"]</code> ）。
<code>dependencies</code>	Record	否	插件依赖定义： 声明运行此插件必须已安装的第三方插件依赖及其版本范围（如 <code>{"ext-roll-call": "^1.0.0"}</code> ）。

2. 能力授权流转 (Capabilities Flow)

清单文件中的 `capabilitiesProposed` 是插件对权限的“申请”。当 `PluginHost` 激活插件时，系统会做如下安全校验：

- 系统会调用 `CapabilityGuard` 将申请的能力与内置的白名单映射进行比对。

- 如果插件申请了不存在于系统白名单中的未知特权，激活流程将**立即阻断报错**（抛出 `InvalidCapabilityError`），插件状态会被标记为 `error` 且拒绝执行，以确保宿主安全。

四、内核服务接口参考与注入对象说明

通过 `ctx.services` 或者 `await ctx.resolve(token)` 可调用的核心服务 API 及接口声明如下：

1. ICommandBusService (指令总线服务)

```
export interface ICommandBusService {  
  // 执行一条系统核心或插件动作指令  
  execute<T = any>(command: {  
    id: string; // 指令唯一运行 ID  
    type: string; // 指令名称类型, 例如 "whiteboard.draw"  
    actorId?: string; // 执行人 (默认插件名)  
    payload: Record<string, any>; // 指令输入参数  
    correlationId?: string; // 关联事件 ID  
  }): Promise<T>;  
  
  // 注册该插件所定义动作的执行逻辑处理器  
  registerHandler(  
    commandType: string,  
    handler: { execute: (command: any) => Promise<any> }  
  ): Promise<void>;  
}
```

2. IActionRegistryService (动作定义注册服务)

```
export interface IActionRegistryService {  
  register(descriptor: {  
    id: string; // 动作标识  
    commandType: string; // 映射指令类型, 必须全局唯一  
    description: string; // 详细的人类与 AI 描述信息 (决定 AI 生成效果的关键)  
    capabilityRequired: string; // 运行该动作所需的安全能力权限, 如 "whiteboard:write"  
    isHighRisk?: boolean; // 是否为高危敏感指令 (会触发审批流拦截)  
    inputSchema: { // 输入参数定义, 采用简化的 JSON Schema 规范  
      type: "OBJECT";  
      properties: Record<string, { type: "STRING" | "NUMBER" | "ARRAY"; description?: string; items?: any }>;  
      required?: string[];  
    };  
  }): Promise<void>;  
}
```

3. IEventBusService (事件发布与订阅总线)

```
export interface IEventBusService {  
  subscribe(eventType: string, handler: (event: any) => void): string; // 返回订阅 UUID 签名  
  unsubscribe(eventType: string, handler: (event: any) => void): void;  
  publish(event: {  
    id: string;  
    type: string;  
    source: string;  
    payload: any;  
    timestamp: number;  
    correlationId?: string;  
  }): Promise<void>;  
}
```


五、 内置指令与系统核心事件目录

1. 内置核心指令参考目录 (Commands Catalog)

指令类型 (type)	权限要求	参数载荷 (payload)	返回数据说明
whiteboard.draw	whiteboard:write	lessonId (string), type ("rectangle", "text" 等), data (图形细节 JSON 串)	返回新生成的画板图形节点标识 { elementId }
whiteboard.update	whiteboard:write	lessonId (string), elementId (string), data (覆盖更新的图形 JSON 串)	{ success: true }
whiteboard.delete	whiteboard:write	lessonId (string), elementId (string)	{ success: true }
vfs.write_file	vfs:write	path : 完整虚拟绝对路径, content : 写入数据字符串	返回新节点主键 { fileId }
vfs.read_file	vfs:read	path : 完整绝对路径	返回文本内容 { content: string }
vfs.list_dir	vfs:read	path : 完整绝对目录路径	返回子项数组 { nodes: any[] }
ai.apply_recommendation	lesson:write	taskType, topic, title, content (均为 string)	⚠️ 高危指令: 写入计划数据库。非管理员用户调用需通过审批流审核。
ai.apply_grade	assignment:write	assignmentId, studentId (string), score (number), feedback (string)	⚠️ 高危指令: 写入成绩记录。非管理员用户调用需通过审批流审核。

2. 内置核心事件清单 (Events Catalog)

插件可以通过订阅以下事件来实现业务联动和数据追踪：

事件类型 (type)	事件源 (source)	事件数据载荷定义 (payload)	触发机制描述
system.ready	core.kernel	{ timestamp: number }	系统内核及 6 个内置默认插件全部引导成功
lesson.started	builtin.lesson	{ lessonId: string, classId: string, teacherId: string, timestamp: number }	教师端点下“开始上课”按键，课堂会话初始化完成
lesson.ended	builtin.lesson	{ lessonId: string, timestamp: number }	教师端下课，课堂结课
student.joined	builtin.classroom	{ lessonId: string, studentId: string, name: string, timestamp: number }	学生端进入课堂会话，在线花名册更新时触发
student.left	builtin.classroom	{ lessonId: string, studentId: string, timestamp: number }	学生网络异常断联或主动离开课堂触发
vfs.file_written	builtin.vfs	{ fileId: string, path: string, timestamp: number }	系统或任意插件写入虚拟文件成功时广播

六、前端 UI 扩展与交互机制

交互式教学系统需要丰富的用户界面。OpenLearn Next V2 通过微前端（MFE）及前端插槽（Extension Slots）向插件提供 UI 扩展能力。

1. 界面扩展插槽 (Extension Slots)

前端拦截系统支持在主应用的以下布局插槽渲染插件的前端组件：

- `teacher.panel`：教师独立全宽管理面板（适用于排课、大屏投票配置管理）。
- `student.fullscreen`：学生全屏视图，会遮罩其他交互（常用于突击测验、考试模式）。
- `classroom.tool`：课堂右下角浮动工具箱（浮动 Dock）按键，点击触发快速操作（如随机点名）。
- `teacher.dashboard.widget`：教师主页看板磁贴小部件。
- `global.setting`：全局设置页扩展。

2. 前端与沙箱 Worker 的 IPC 通信协议

前端 UI 属于浏览器主线程，而后端插件逻辑运行在隔离的 Worker 线程沙箱中。它们之间的双向通信架构如下：



1. **前端下发控制**: 前端组件通过微前端上下文 `useMfeContext()` 消费平台能力，向 `eventBus` 发布事件，或调用后台注册的插件扩展 Command。
2. **后端更新反馈**: 插件后端计算完成后，调用 `whiteboard.update` 更新画板元素，或通过事件广播触发前端组件重新渲染。

3. 内置 UI 统一规范组件库

为保证视觉一致性，宿主平台提供了规范 of UI 组件集。开发者可在子应用中导入以下标准组件：

- `<Button variant="primary" size="md">`：带有平台微交互的统一色值按钮。
- `<FormGroup label="选项名称">`：支持数据绑定的响应式表单项组。
- `<Card title="数据统计" shadow="sm">`：统一圆角及阴影的标准面板卡片。

七、隔离数据库设计与构建

1. 物理表空间隔离与 ctx.db 映射

物理表名由系统自动加上插件 UUID 作为前缀并规范重映射，保证完全杜绝 SQL 注入与其他插件的越权访问：

```
物理表名 = plugin_<pluginId_to_underscores>_<tableName>
```

2. 数据迁移策略 (Database Schema Migrations)

当插件从 `v1.0.0` 升级至 `v2.0.0` 时，若有新增数据列需求，由于 `ensureTable` 无法自动变更已存在的表结构，开发者必须在 `activate` 周期中手动进行 SQLite 兼容迁移：

```
// 数据库结构手动增列迁移示例
activate: async (ctx) => {
  const db = await ctx.resolve<any>(IDatabaseToken);
  const tblVotes = ctx.db.table('votes');

  // 1. 确保基础表存在
  await ctx.db.ensureTable('votes', 'id TEXT PRIMARY KEY, lesson_id TEXT, title TEXT');

  // 2. 检测字段是否存在，动态执行 ALTER TABLE
  const tableInfo = await db.prepare("PRAGMA table_info(" + tblVotes + ")").all();
  const hasElementIds = tableInfo.some((col: any) => col.name === 'element_ids');

  if (!hasElementIds) {
    // 动态添加新列
    await db.prepare("ALTER TABLE " + tblVotes + " ADD COLUMN element_ids TEXT").run();
    console.log("[Migration] Successfully added column 'element_ids' to " + tblVotes);
  }
}
```

3. 插件卸载时的数据回收机制

当用户在系统控制台中对插件执行“卸载”操作时，系统提供以下两种结构清理策略：

- **默认清理 (Hard Purge):** 清理引擎会自动检索并运行插件注册下的所有物理表，依次执行 `DROP TABLE IF EXISTS plugin_<id>_tableName`，防止存储碎片堆积。

- **保留数据 (Soft Keep):** 如果插件清单中配置了保留选项（待开发内置属性），物理表数据将得以保留，以便在插件重新安装时实现数据恢复。

八、沙箱限制、错误处理与调试指南

1. 运行资源限制与配额 (Quotas & Limits)

为避免耗尽系统资源导致内核锁死，插件 Worker 进程会受到以下硬性指标配额约束：

- **内存上限 (Memory Limit):** 每个 Worker 线程的最大 V8 堆内存被限制为 **128 MB**（超出此配额会导致线程 OOM 崩溃并触发内核重启机制）。
- **执行时间片 (CPU Execution Timeout):** 每次执行 Command 或 Action 处理器的阻塞耗时阈值为 **10 秒**。超时仍未返回的调用会被内核强行终止并抛出超时异常。
- **磁盘及存储配额 (Storage Limit):** VFS 文件系统单插件最大写入配额为 **50 MB**。隔离 SQLite 数据库单表的最大记录上限为 **10,000 行**。

2. 异常捕获与日志管理

- **错误捕获:** 插件执行中抛出的未捕获错误会被沙箱边界捕获，向主线程返回 `commandError` 并记录对应的错误堆栈 (Stack Trace)。插件不应吞掉核心异常，以便主线程感知。
- **日志捕获:** 沙箱内重写了全局 `console.log` 及 `console.error`。所有输出都会被打上 `[Worker stdout - <UUID>]` 前缀，自动管道传输重定向写入至系统运行日志文件中，路径为：
`<appDataDir>/brain/<conversationId>/system_generated/tasks/task-xxx.log`。

3. 断点调试指南 (Debugging)

调试多线程沙箱插件需要借助 Node 调试端口映射：

1. 启动内核时，增加参数允许 Worker 调试：

```
node --inspect-brk packages/core/index.js
```

2. 打开 Chrome 浏览器，访问 `chrome://inspect`，在 Remote Target 中找到该进程并连接。
3. 在 `WorkerManager` 动态生成的 base64 data URL 节点处打下断点，即可实时监视 Worker 内部的执行状态和 RPC 数据包流转。

九、完整的 Demo 示例：Raffle-Vote 插件

以下为一个完整的第三方全栈插件代码。该插件包含画板图形绘制、数据库自建表操作、事件流发布、以及利用共享模块 `xlsx` 导出 Excel 明细数据的功能。

1. 清单文件 manifest.json

```
{
  "id": "ext-raffle-vote",
  "name": "课堂实时投票与抽奖转盘",
  "version": "1.0.0",
  "description": "提供课堂互动投票与白板条形图展示，支持幸运学生随机点名并导出汇总报表",
  "entry": "index.js",
  "capabilitiesProposed": [
    "whiteboard:write",
    "vfs:write"
  ]
}
```


2. 主入口文件 index.ts

```
import { IDatabaseToken } from '../core/di/interfaces.js';
import type { PluginContext } from '../core/plugin-host/types.js';

export default {
  manifest: {
    id: "ext-raffle-vote",
    name: "课堂实时投票与抽奖转盘",
    version: "1.0.0"
  },

  activate: async (ctx: PluginContext) => {
    const commandBus = ctx.services.commandBus!;
    const actionRegistry = ctx.services.actionRegistry!;

    console.log("[Raffle & Vote Plugin] Activating plugin " + ctx.pluginId + "...");

    // 1. 创建隔离的数据表
    await ctx.db.ensureTable('votes', 'id TEXT PRIMARY KEY, lesson_id TEXT, title TEXT, options TEXT, element TEXT');
    await ctx.db.ensureTable('votes_cast', 'vote_id TEXT, option_index INTEGER, voter_id TEXT, timestamp INTEGER');

    // 2. 注册开始投票 Action (vote.create)
    await actionRegistry.register({
      id: 'ext-vote-create',
      commandType: 'vote.create',
      description: '在课堂白板上发起一个实时的多选投票，渲染为动态条形图',
      capabilityRequired: 'whiteboard:write',
      inputSchema: {
        type: 'OBJECT',
        properties: {
          lessonId: { type: 'STRING', description: '课程 ID' },
          title: { type: 'STRING', description: '投票题目' },
          options: { type: 'ARRAY', items: { type: 'STRING' } }
        },
        required: ['lessonId', 'title', 'options']
      }
    });

    // 3. 注册投下选票 Action (vote.cast)
    await actionRegistry.register({
      id: 'ext-vote-cast',
      commandType: 'vote.cast',
      description: '学生投下一票，实时更新白板的条形图高度',
      capabilityRequired: 'whiteboard:write',
      inputSchema: {
        type: 'OBJECT',
        properties: {
          lessonId: { type: 'STRING' },
          voteId: { type: 'STRING' },
          optionIndex: { type: 'NUMBER' },
          voterId: { type: 'STRING' }
        }
      }
    });
  }
};
```

```

    },
    required: ['lessonId', 'voteId', 'optionIndex', 'voterId']
  }
});

// 4. 注册导出投票结果 Action (vote.export)
await actionRegistry.register({
  id: 'ext-vote-export',
  commandType: 'vote.export',
  description: '将当前投票汇总导出为 Excel 存入 VFS 虚拟文件系统',
  capabilityRequired: 'vfs:write',
  inputSchema: {
    type: 'OBJECT',
    properties: {
      voteId: { type: 'STRING' }
    },
    required: ['voteId']
  }
});

// — 5. 绑定 vote.create 处理器 —————
await commandBus.registerHandler('vote.create', {
  execute: async (command) => {
    const db = await ctx.resolve<any>(IDatabaseToken);
    const { lessonId, title, options } = command.payload as any;
    const voteId = 'vote_' + Math.random().toString(36).slice(2, 10);
    const x = 150, y = 150;

    const bgHeight = 65 + options.length * 45;
    const bgDraw = await commandBus.execute({
      id: 'draw_bg_' + voteId,
      type: 'whiteboard.draw',
      payload: {
        lessonId, type: 'rectangle',
        data: JSON.stringify({ x, y, width: 380, height: bgHeight, fill: '#f8fafc', stroke: '#cbd5e1',
      }
    }) as any;

    const optionElementIds: any[] = [];
    for (let i = 0; i < options.length; i++) {
      const barDraw = await commandBus.execute({
        id: "draw_bar_" + voteId + "_" + i,
        type: 'whiteboard.draw',
        payload: { lessonId, type: 'rectangle', data: JSON.stringify({ x: x + 20, y: y + 50 + i * 45, w
      }) as any;

      const labelDraw = await commandBus.execute({
        id: "draw_label_" + voteId + "_" + i,
        type: 'whiteboard.draw',
        payload: { lessonId, type: 'text', data: JSON.stringify({ x: x + 20, y: y + 50 + i * 45 + 20, t
      }) as any;

      optionElementIds.push({ barId: barDraw?.elementId, labelId: labelDraw?.elementId });
    }
  }
});

```

```

const tblVotes = ctx.db.table('votes');
await db.prepare("INSERT INTO " + tblVotes + " (id, lesson_id, title, options, element_ids, created
  voteId, lessonId, title, JSON.stringify(options), JSON.stringify({ bgId: bgDraw?.elementId, optio
));

return { success: true, voteId };
}
});

// — 6. 绑定 vote.cast 处理器 —————
await commandBus.registerHandler('vote.cast', {
  execute: async (command) => {
    const db = await ctx.resolve<any>(IDatabaseToken);
    const { lessonId, voteId, optionIndex, voterId } = command.payload as any;

    const tblVotes = ctx.db.table('votes');
    const tblCast = ctx.db.table('votes_cast');

    const voteConfig = await db.prepare("SELECT * FROM " + tblVotes + " WHERE id = ? AND lesson_id = ?"
    if (!voteConfig) throw new Error('找不到指定的投票活动');

    const options = JSON.parse(voteConfig.options);
    const elementIds = JSON.parse(voteConfig.element_ids);

    const checkVoted = await db.prepare("SELECT count(*) as count FROM " + tblCast + " WHERE vote_id =
    if (checkVoted.count > 0) throw new Error('不可重复投票');

    await db.prepare("INSERT INTO " + tblCast + " (vote_id, option_index, voter_id, timestamp) VALUES (

    const allVotes = await db.prepare("SELECT option_index, count(*) as count FROM " + tblCast + " WHERE
    const counts: Record<number, number> = {};
    options.forEach((_: any, i: number) => { counts[i] = 0; });
    let total = 0;
    allVotes.forEach(v => { counts[v.option_index] = v.count; total += v.count; });

    for (let i = 0; i < options.length; i++) {
      const count = counts[i];
      const pct = total > 0 ? (count / total) : 0;
      const barWidth = Math.max(15, pct * 280);
      const refs = elementIds.options[i];

      await commandBus.execute({
        id: 'update_bar_' + Math.random().toString(36).slice(2),
        type: 'whiteboard.update',
        payload: { lessonId, elementId: refs.barId, data: JSON.stringify({ x: elementIds.x + 20, y: ele
      });

      await commandBus.execute({
        id: 'update_label_' + Math.random().toString(36).slice(2),
        type: 'whiteboard.update',
        payload: { lessonId, elementId: refs.labelId, data: JSON.stringify({ x: elementIds.x + 20, y: e
      });
    }
  }
}

```

```

        return { success: true, totalVotes: total };
    }
});

// — 7. 绑定 vote.export 处理器 —————
await commandBus.registerHandler('vote.export', {
    execute: async (command) => {
        const db = await ctx.resolve<any>(IDatabaseToken);
        const { voteId } = command.payload as any;
        const tblVotes = ctx.db.table('votes');
        const tblCast = ctx.db.table('votes_cast');

        const vote = await db.prepare("SELECT * FROM " + tblVotes + " WHERE id = ?").get(voteId) as any;
        const votesCast = await db.prepare("SELECT * FROM " + tblCast + " WHERE vote_id = ? ORDER BY timest

        const options = JSON.parse(vote.options);
        const counts: Record<number, number> = {};
        options.forEach((_: any, i: number) => { counts[i] = 0; });
        let total = 0;
        votesCast.forEach(v => { counts[v.option_index]++; total++; });

        const fn = "vote_results_" + voteId + ".xlsx";
        let xlsMod: any = null;
        try {
            xlsMod = ctx.require('xlsx');
        } catch {
            console.warn('xlsx 模块不可用, 降级为 CSV 格式导出');
        }

        if (xlsMod) {
            const wb = xlsMod.utils.book_new();
            const summaryRows = options.map((opt: string, i: number) => ({
                '选项': opt,
                '得票数': counts[i],
                '占比': total > 0 ? Math.round((counts[i] / total) * 100) + "%" : '0%'
            }));
            xlsMod.utils.book_append_sheet(wb, xlsMod.utils.json_to_sheet(summaryRows), '投票汇总');

            const detailRows = votesCast.map(v => ({
                '投票人ID': v.voter_id,
                '所选选项': options[v.option_index],
                '投票时间': new Date(v.timestamp).toISOString()
            }));
            xlsMod.utils.book_append_sheet(wb, xlsMod.utils.json_to_sheet(detailRows), '投票明细');

            const excelBuffer = xlsMod.write(wb, { type: 'buffer', bookType: 'xlsx' });
            await commandBus.execute({
                id: 'export_vfs_excel_' + Math.random().toString(36).slice(2),
                type: 'vfs.write_file',
                payload: { path: "/exports/" + fn, content: excelBuffer.toString('base64') }
            });
            return { success: true, path: "/exports/" + fn, format: 'xlsx' };
        } else {

```

```
        return { success: true, format: 'csv' };  
    }  
    }  
    });  
},  
  
deactivate: async () => {  
    console.log('[Raffle & Vote Plugin] Deactivating plugin...');  
}  
};
```

十、编译打包与集成测试

1. 自动化打包编译脚本

```
import esbuild from 'esbuild';
import archiver from 'archiver';
import fs from 'fs';

await esbuild.build({
  entryPoints: ['packages/plugins/raffle-vote/index.ts'],
  bundle: true,
  outfile: 'dist/temp/index.js',
  format: 'esm',
  external: ['xlsx', 'uuid', 'jspdf'],
  platform: 'node'
});

const output = fs.createWriteStream('dist/plugins/ext-raffle-vote.zip');
const archive = archiver('zip', { zlib: { level: 9 } });
archive.pipe(output);
archive.file('dist/temp/index.js', { name: 'index.js' });
archive.file('packages/plugins/raffle-vote/manifest.json', { name: 'manifest.json' });
await archive.finalize();
console.log('Zip 插件打包完成。');
```

2. 编写 Vitest 集成测试

```
import { describe, it, expect, beforeAll, afterAll } from 'vitest';
import { Kernel } from '../core/kernel/index.js';

describe('Raffle & Vote Plugin Test', () => {
  let kernel: Kernel;

  beforeAll(async () => {
    const { db } = await import('../core/db/index.js');
    db.prepare("DELETE FROM plugins WHERE manifest LIKE '%ext-raffle-vote%'").run();
    kernel = new Kernel();
    await kernel.ready;
  });

  it('应该成功发起投票并能够投下选票', async () => {
    const createResult = await kernel.commandBus.execute({
      id: 'cmd-vote-create',
      type: 'vote.create',
      actorId: 'user-teacher',
      payload: {
        lessonId: 'test-lesson-id',
        title: '你喜欢 JavaScript 吗?',
        options: ['非常喜欢', '讨厌']
      }
    }) as any;
    expect(createResult.success).toBe(true);

    const castResult = await kernel.commandBus.execute({
      id: 'cmd-vote-cast',
      type: 'vote.cast',
      payload: {
        lessonId: 'test-lesson-id',
        voteId: createResult.voteId,
        optionIndex: 0,
        voterId: 'student-alice'
      }
    }) as any;
    expect(castResult.success).toBe(true);
    expect(castResult.totalVotes).toBe(1);
  });
});
```

十一、版本迭代、发布与分发流程

1. 生产审核与状态流转机制

当管理员在上架面板上传打包后的 `.zip` 插件包时，系统会将其注册并置为 `installed` 状态。激活操作由内核调度的 `ready` 流水线管理，在成功注入授权及启动 Worker 实例后转为 `active` 状态。遇到崩溃或加载失败则进入 `error` 挂起状态并写入事件流通知。

2. 无停机热更新机制 (Hot Swapping)

内核对处于 Worker 隔离状态下的插件支持热交换热更新：

1. 上传新版 `.zip` 文件后，主线程通过 `WorkerManager` 派生一个新的独立 Worker 实例加载新代码。
2. 新 Worker 实例初始化并加载成功（状态转为 `active`）。
3. 主线程路由器将后续的所有 Action 指令路由重定向至新 Worker。
4. 旧 Worker 中的存量待处理指令处理完毕后，系统通过 `deactivate` 清除其关联事件总线订阅并销毁线程，实现零停机平滑过度。

3. 系统待开发功能清单 (Feature Backlog)

为了让该插件生态系统更加完备，开发团队已将以下生产级特性列入后续迭代的 **Backlog 待办列表**。欢迎社区开发者共同贡献：

优先级	功能模块	功能描述	设计草案与预留 API 接口
P1	内置数据库迁移 API	提供统一的高级表变更接口，无需开发者在 <code>activate</code> 里手写 SQLite 变更检测语句。	设计提供 <code>ctx.db.migrate(version, (db) => { ... })</code> 以进行安全结构版本递增管理。
P1	微前端热更状态继承	热替换升级插件时，允许新旧 Worker 传递并交接内存缓存中的运行状态，防止断线或重置。	设计在 <code>deactivate(state)</code> 导出序列化状态，并在新 Worker 激活时由 <code>activate(ctx, state)</code> 还原。
P2	硬性 CPU 时间片控制	限制单个沙箱 Worker 的高频密集计算，避免抢占宿主 CPU 周期。	设计在 Worker 引导模板中注入基于 <code>performance.now()</code> 调度的循环限制器（CPU throttling）。
P2	插件分发数字签名校验	确保上架的第三方 Zip 插件没有在网络分发中被篡改或带毒。	在上架和安装接口中，采用基于内核 RSA 公钥解密的 <code>manifest.signature</code> 电子签名强校验。